

Universidad del Valle de Guatemala
Facultad de Ingeniería — Ciencias de la Computación

Construcción de Compiladores Compilador

Análisis Semántico de Compiscript
Generador de Analizadores Semánticos

Integrantes

Isabella Obando – 23074

Anthony Lou Schwank – 23410

Juan Francisco Martinez – 23617

Repositorio de GitHub

<https://github.com/Isabella1911/Compis>

Índice

1. Descripción del proyecto	2
2. Objetivos	2
2.1. Generales	2
2.2. Específicos	2
3. Arquitectura de la implementación	3
3.1. Pipeline	3
3.2. Estructura del repositorio	3
3.3. Entrada y salida	3
4. Tabla de símbolos	4
5. Reglas semánticas implementadas	4
5.1. Sistema de tipos	4
5.2. Manejo de ámbito	5
5.3. Funciones y procedimientos	5
5.4. Control de flujo	5
5.5. Clases y objetos	5
5.6. Listas y generales	6
6. Funcionamiento del programa	6
7. IDE	6
8. Batería de pruebas	7
9. Cómo compilar y ejecutar	7
9.1. Primera vez (sin sudo)	7
9.2. Uso habitual	7
10. Decisiones de lenguaje relevantes	7

1. Descripción del proyecto

Esta actividad corresponde a la fase de **análisis semántico** en la construcción de un compilador para **Compiscript**, un lenguaje basado en un subconjunto de TypeScript. Se implementó un analizador sintáctico a partir de la gramática oficial en **ANTLR** y se extendió con acciones semánticas: se construye un árbol sintáctico con representación visual y se recorre mediante un **Visitor** de ANTLR (transformación a AST propio) seguido de varios pases semánticos sobre ese AST.

Analizador sintáctico

Basado en `grammar/Compiscript.g4` (ANTLR 4), generando lexer y parser C++. El `Visitor AstBuilder` produce un AST propio desacoplado de los tipos de ANTLR.

Análisis semántico

Verificación de tipos, ámbitos, funciones, control de flujo, clases y estructuras de datos sobre el AST.

Tabla de símbolos

Estructura central con entornos anidados (global, función, clase, bloque) que acompaña todas las fases del análisis.

2. Objetivos

2.1. Generales

- Implementar un analizador semántico funcional para Compiscript.
- Aplicar sistemas de tipos, ámbitos y entornos sobre un árbol sintáctico real.
- Diseñar una tabla de símbolos capaz de sostener fases posteriores del compilador.

2.2. Específicos

- Analizador sintáctico con ANTLR a partir de la gramática oficial.
- Árbol sintáctico con representación visual (texto indentado y DOT/Graphviz).
- Recorrido del árbol (`Visitor ANTLR` → AST; pases semánticos sobre el AST) para evaluar las reglas del enunciado.
- Batería de pruebas con casos exitosos y fallidos por regla semántica.
- IDE para escribir, cargar y analizar código `.cps`.

3. Arquitectura de la implementación

3.1. Pipeline

```
.cps
-> ANTLR (lexer + parser)
-> AstBuilder (Visitor) -> AST propio
-> DeclarationCollector
-> InheritanceResolver
-> NameResolver
-> TypeChecker (preparacion de firmas + validacion)
-> ControlFlowChecker
-> ClosureAnalyzer
-> CompilationResult { ast, diagnostics, symbol_table }
```

La fachada pública es `Compiler::compile(source)`. El CLI (`compiscript`) e IDE invocan el mismo binario. Un error léxico/sintáctico impide construir el AST; los errores semánticos permiten continuar los pases para acumular diagnósticos independientes.

3.2. Estructura del repositorio

Ruta	Responsabilidad
<code>grammar/Compiscript.g4</code>	Gramática oficial (sin modificar)
<code>src/frontend/</code>	Único módulo que incluye headers ANTLR
<code>src/ast/</code>	Nodos del AST, impresión texto/DOT
<code>src/diagnostics/</code>	Códigos SYN/SEM, reporter
<code>src/compiler/</code>	Fachada <code>Compiler::compile</code>
<code>src/semantic/</code>	Símbolos, tipos y pases semánticos
<code>ide/</code>	IDE Python/Tkinter sobre el binario
<code>tests/fixtures/</code>	Casos válidos e inválidos por regla
<code>tools/setup.sh</code>	Toolchain reproducible sin <code>sudo</code>

3.3. Entrada y salida

Entrada

Archivo fuente Compiscript con extensión `.cps`.

Salida ■ Representación visual del AST (texto + `output/ast.dot`).

- Reporte de errores semánticos (código, severidad, línea, columna, mensaje).
- Estado de la tabla de símbolos por entorno (global, función, clase, bloque).

4. Tabla de símbolos

La tabla se implementa como **pila de tablas** (Scope con padre e hijos):

- `Symbol` / `FunctionSymbol` / `ClassSymbol`.
- `Scope::declare` / `resolve` / `resolveLocal`.
- Un entorno nuevo por función, clase y bloque.
- Shadowing entre ámbitos hijos; redeclaración en el mismo ámbito → SEM002.
- Símbolos rechazados se conservan para recuperación sin contaminar búsquedas.

5. Reglas semánticas implementadas

A continuación se mapean los requisitos del enunciado a códigos de diagnóstico y al pase responsable. La batería exige el **código exacto** en `tests/fixtures/expected.json`.

5.1. Sistema de tipos

Regla del PDF		Código	Notas
Operaciones	aritméticas	SEM004	Operandos <code>integer</code> (no hay <code>float</code> en la gramática)
+, −, *, /			
Operaciones lógicas	&&, , !	SEM004	Operandos <code>boolean</code>
Comparaciones		SEM004	Compatibilidad de operandos
Asignaciones vs tipo declarado		SEM003	También destino no asignable / <code>const</code>
Inicialización de <code>const</code>		SYN001	Exigida por la gramática
Listas / estructuras		SEM004, SEM018, SEM019	Elementos, <code>foreach</code> , inferencia

5.2. Manejo de ámbito

Regla	Código	Pase
Resolución local/global	—	NameResolver
Variable no declarada	SEM001	NameResolver
Redeclaración mismo ámbito	SEM002	DeclarationCollector
Bloques anidados	—	Scope por bloque
Nuevo entorno función/clase/- bloque	—	DeclarationCollector

5.3. Funciones y procedimientos

Regla	Código	Notas
Número/tipo de argumentos	SEM008	Posicional; métodos y constructor
Tipo de retorno	SEM009, SEM020	Valor incompatible; camino sin return
Recursión	—	Firmas preparadas antes de cuerpos
Funciones anidadas / closures	—	ClosureAnalyzer llena capturas
Redeclaración de funciones	SEM002	
Parámetro sin tipo	SEM011	Decisión de lenguaje

5.4. Control de flujo

Regla	Código	Notas
Condiciones booleanas	SEM005	if/while/do-while/for /ternario
break/continue fuera de bucle	SEM006	switch no cuenta como bucle
return fuera de función	SEM007	
Código muerto	SEM012	Tras return/break/continue

5.5. Clases y objetos

Regla	Código	Notas
Atributos/métodos vía .	SEM010	Incluye herencia
Invocación de constructor	SEM008, SEM017	Nombre constructor
this	SEM015	Fuera de clase
Herencia inválida / circular	SEM013, SEM014	

5.6. Listas y generales

- Tipo de elementos e índices: SEM004; `foreach` sobre no-arreglo: SEM018.
- Expresiones sin sentido (p. ej. invocar no-función): SEM016.
- Declaraciones duplicadas (variables, parámetros): SEM002.

6. Funcionamiento del programa

1. El usuario carga o escribe un `.cps` en la IDE (`python3 ide/ide_app.py`).
2. ANTLR construye el parse tree; `AstBuilder` genera el AST propio (texto + DOT/PNG).
3. Los pases semánticos recorren el AST, construyen/consultan la tabla de símbolos por entorno y emiten diagnósticos.
4. Se reportan errores con tipo (código), ubicación y mensaje; la IDE resalta líneas con error.
5. La batería se ejecuta con `make test`.

7. IDE

Componente de evaluación: **15 pts**.

- Editor para `.cps`: abrir, editar, guardar.
- **Compilar** construye el binario (`make`).
- **Ejecutar análisis** invoca `./compiscript` como subproceso.
- Pestañas: diagnósticos (código, línea, columna, mensaje), AST texto, tabla de símbolos, AST gráfico (Graphviz).
- Doble clic en un diagnóstico salta a la posición; las líneas con error se resaltan.

Detalle de uso: `ide/IDE.md`.

8. Batería de pruebas

Requisito explícito del enunciado: casos exitosos y fallidos por regla, presente al momento de la evaluación.

- Fixtures en `tests/fixtures/valid/` e `tests/fixtures/invalid/`.
- Nombres por regla (`sem004_tipo_aritmetica.cps`, ...).
- `expected.json` exige el conjunto exacto de códigos (`SEM0xx/SYN0xx`), no solo “hubo un error”.
- `make test`: unitarios de símbolos/semántica, runner y fixtures (timeout 5 s, sin stderr inesperado).

9. Cómo compilar y ejecutar

9.1. Primera vez (sin sudo)

```
bash tools/setup.sh
```

Instala JRE, `antlr.jar`, CMake portátil y el runtime C++ de ANTLR en `.deps/`.

9.2. Uso habitual

```
make
make run FILE=tests/fixtures/valid/hello.cps
make test
python3 ide/ide_app.py
```

El binario imprime diagnósticos, AST, tabla de símbolos y exporta `output/ast.dot`. Renderizado opcional:

```
dot -Tpng output/ast.dot -o ast.png
```

10. Decisiones de lenguaje relevantes

1. No existe `float` en `Compiscript.g4`; solo `boolean`, `integer`, `string` e identificadores de clase.
2. Parámetros requieren anotación semántica (`SEM011`).
3. Parámetro de `catch`: tipo interno `string`.

4. `string + integer` no permitido; `+` es `entero+entero` o `string+string`.
5. `if/while/for/...` exigen bloque con llaves (según la gramática).
6. Constructor: método llamado literalmente `constructor`.